

A Heuristic Load-Balancing Algorithm for Multi-Mesh Simulations on GPUs

Leo Sitaraman*

Homestead High School, Cupertino, CA

An algorithm for partitioning and load balancing multiple grids for GPU CFD solvers is presented. The algorithm targets scenarios where a small number of processing elements are used to handle a large number of grids with disparate sizes. Unlike traditional CPU-based solvers, which often rely on thousands of processing elements, modern GPU solvers typically use only a small number (1–32), as a single modern GPU can provide performance comparable to several hundred CPU cores. Existing partitioning algorithms were primarily developed for CPU architectures and focus on distributing a small number of grids across a large number of processors. In contrast, the algorithm introduced here addresses the inverse problem by efficiently distributing many grids across few processing elements. Performance results demonstrate 2-3x speedup compared to a naïve uniform partitioning scheme. The implementation is released as open-source software under the MIT license.

I. Introduction

Mesh generation and partitioning are fundamental components of physics-based numerical simulations across disciplines such as fluid dynamics, structural mechanics, and electromagnetics. Many practical aerospace simulations rely on multiple meshes to represent complex geometries composed of interacting bodies. The overset grid method, in which each body is discretized using its own mesh and the resulting grids overlap in space, provides flexibility and modularity in mesh generation. Overset meshes are particularly advantageous for problems involving relative body motion, moving control surfaces, or elastic deformation, as they simplify simulation setup and enable the use of grid types tailored to individual components. An example of an overset mesh system from Ref. [1] is shown in Fig. 1, where several meshes of different sizes are used in the near-body (NB) region and are immersed in an off-body (OB) mesh system with nested refinement zones. The overset grid method has been successfully applied to a wide range of aerospace problems, including rotorcraft flow fields, aircraft with moving control surfaces, store separation, and wind turbine simulations.

Modern physics simulation codes (Refs. [2–6]) increasingly target graphics processing units (GPUs) to reduce time-to-solution while maintaining engineering-level accuracy. GPU architectures offer massive fine-grained parallelism and performance equivalent to hundreds or thousands of CPU cores, often enabling speedups of five to ten times compared to CPU-only execution. Achieving these gains in multi-mesh simulations, however, requires effective load balancing of meshes across available processing elements (PEs).

Load balancing is typically achieved through domain decomposition, in which each mesh is subdivided into smaller computational subdomains. These domains are distributed among PEs using methods that attempt to minimize the maximum load assigned to any single PE. Classical domain decomposition strategies were developed primarily for CPU-based systems with large numbers of PEs and relatively low communication costs. A commonly used approach is binned grouping [7], in which meshes are first over-decomposed into many small blocks and then distributed to PEs to balance workload. While effective in CPU-centric environments, excessive decomposition increases the number of inter-block interfaces, leading to higher communication overhead and reduced overall efficiency. To mitigate over-decomposition that arises from binned-grouping, Sherer and Garmann developed the BUNGE code [8], which uses a recursive search algorithm to perturb an initial pro-rated partitioning and improve load balance while limiting unnecessary splits. Their approach focused on structured grids and imposed the constraint that each subdivision retain a structured, array-based form. Performance improvements of up to 40 percent were demonstrated for simulations employing more than 200 processing elements. These results were achieved under the assumption that the number of meshes was much smaller than the number of PEs, enabling each PE to process a single subdomain from one mesh.

*11th grade student, Homestead high school, Student Member 1925964

This assumption no longer holds for modern GPU-based systems. A typical GPU node contains a small number of highly capable processing elements, and the number of PEs is often comparable to, or smaller than, the number of meshes in an overset simulation. In this regime, it is neither feasible nor efficient to assign only a single mesh or subdomain to each PE. Furthermore, the relative cost of communication compared to computation is significantly higher on GPUs, making it especially important to minimize mesh splits and inter-PE communication. These architectural considerations motivate the need for load-balancing strategies specifically designed for GPU-based multi-mesh simulations. Such strategies must balance computational workload while minimizing the number of mesh subdivisions and performing robustly when the number of meshes is comparable to the number of processing elements. This work presents a new heuristic mesh partitioning algorithm developed to address these requirements in GPU-based overset grid simulations.

Specifically, the objective of this work is to develop a new mesh partitioning algorithm that simultaneously minimizes overall load imbalance, defined as the ratio of ideal load to maximum load, and reduces the number of splits required for each mesh. The algorithm should perform adequately when $N \sim P$, where N is the number of meshes and P is the number of PEs.

Because GPU-oriented partitions typically assign multiple subdomains to each PE, the order in which these subdomains are processed becomes important. Subdomains originating from the same mesh require communication with other PEs that hold complementary partitions. Without careful organization, a PE may be forced to pause execution while waiting for other PEs to complete earlier computations. To avoid these delays, the proposed algorithm produces partitions in which each PE contains some undivided meshes and a single group of divided meshes. Undivided meshes are not subdivided and can be processed independently without communication. Each group of divided meshes appears in two or more PEs, and all groups across PEs are disjoint. If a group appears in four PEs, for example, each PE is assigned one-fourth of every mesh in that group. This grouping format ensures simple execution ordering and minimizes communication delays. Each PE first processes the subdomains in its group, followed by its undivided meshes. When executing a group, all PEs involved are guaranteed to be processing the same set of meshes simultaneously, eliminating the need for synchronization stalls due to mismatched execution order. The algorithm seeks partitions of this form that minimize the number of mesh subdivisions while ensuring that the maximum load across all PEs is within a specified tolerance of the ideal load (defined as the total computational work divided by the number of PEs and represents the best possible maximum load under perfect balance). To achieve this, the algorithm first performs a greedy distribution to identify undivided meshes and then iteratively merges and subdivides groups of meshes until the tolerance is satisfied. The algorithm has been implemented and published on github here: <https://github.com/lsitaraman677/Optimal-mesh-distribution>

The algorithm has been integrated into the modern high-performance computational framework CREATETM-AV Helios [4] and has demonstrated significant performance improvements for representative overset grid simulations executed on GPU hardware. These gains are attributed to a reduction in communication overhead, improved utilization of GPU resources, and more efficient assignment of mesh workloads. The algorithm is evaluated using a series of canonical test cases that systematically vary the number of meshes, mesh size distributions, and number of processing elements. These studies assessed scalability, load balance quality, and sensitivity to problem configuration, with particular emphasis on cases in which the number of meshes exceeds or closely matches the number of available processing elements.

Overall, this work addresses a key limitation in existing domain decomposition strategies for overset grid simulations on GPU architectures and provides a practical approach for achieving efficient, scalable load balancing in modern multi-mesh physics simulations.

II. Methodology

The problem considered is to find the optimal solution for splitting N meshes $M = \{M_1, \dots, M_N\}$ with sizes $S = \{S_1, \dots, S_N\}$ into P processing elements. The output of the algorithm is a partitioning list given by $L = \{B_1, B_2, B_3, \dots, B_P\}$, where L is a list of bins with size P , where the bins are defined as $B_i \subseteq M \quad \forall i \in [1, P]$. The number of splits or cuts C_j for each mesh M_j is the equal to the number of times M_j occurs in the multi-set union $A = \biguplus_{i=1}^P B_i$, i.e. $C_j = \text{mult}_A(M_j)$. Given these definitions, several partitioning strategies used in the overset-grid literature [2–4, 7, 8] can be described as follows.

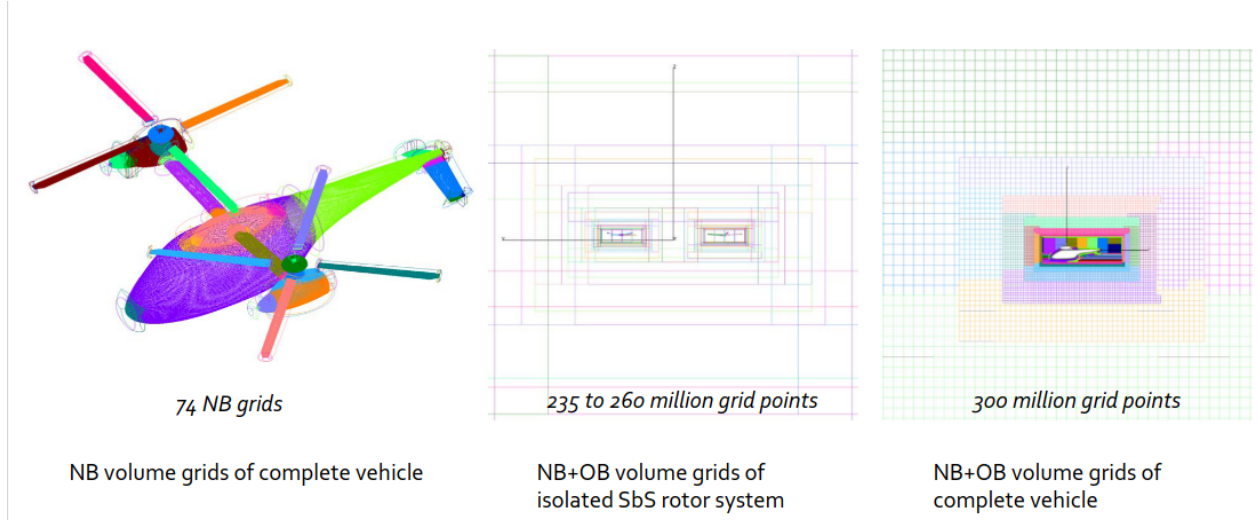


Fig. 1 Overset grid system with near-body (NB) and off-body meshes (OB). Figure from Ref [1]

1) **Uniform partitioning**

Uniform partitioning is the naive approach where

$$B_i = M \quad \forall i \in [1, P].$$

All meshes are split into P parts and each PE processes N partitions, one from each mesh. This approach is perfectly load-balanced since every processor receives identical work. However, it also produces the maximum number of mesh splits, resulting in significant communication overhead.

Furthermore, GPUs typically require sufficiently large mesh sizes (often millions of elements) to achieve good performance. Because uniform partitioning fragments every mesh, it tends to be inefficient for GPU-based simulations.

2) **Greedy partitioning**

For greedy partitioning,

$$B_i \cap B_j = \emptyset \quad \forall B_i, B_j \in L.$$

Each mesh is assigned entirely to a single processor. Meshes are sorted by size and iteratively placed into the least-loaded bin.

This approach introduces no mesh splits and therefore requires minimal communication during the simulation. However, it can suffer from load imbalance when mesh sizes vary significantly.

3) **Linear partitioning**

In linear partitioning, each bin contains a portion of only one mesh,

$$|B_i| = 1 \quad \forall i \in [1, P].$$

The number of cuts for mesh M_j is typically chosen in proportion to mesh size:

$$C_j = \text{round} \left(\frac{S_j}{\sum_{k=1}^N S_k} P \right).$$

The values C_j must be adjusted so that

$$\sum_{j=1}^N C_j = P.$$

This approach is used in the follow-up work of Sherer and Garmann [9], where the cut counts are further optimized using domain decomposition metrics. However, linear partitioning becomes infeasible when $N > P$.

In most practical simulations, particularly for overset grids, N is often comparable to or larger than P . The search space of possible assignments is $O(N^P)$, making exhaustive optimization impractical. Consequently, efficient heuristics are required to produce good solutions with respect to both load balance and the number of mesh cuts.

A. Load Balance and Cut Metrics

The ideal load per processor is

$$IdealLoad = \frac{\sum_{i=1}^N S_i}{P}, \quad maxLoad = \max_i \sum_k S(B_{ik}), \quad (1)$$

where B_{ik} denotes the k -th mesh assigned to processor i . The load imbalance is then defined as

$$LBC = \frac{IdealLoad}{maxLoad} \quad (2)$$

Note that if the load is perfectly balanced, LBC will be equal to one. The total number of mesh splits is

$$N_{cuts} = \sum_{j=1}^N C_j. \quad (3)$$

The objective of the proposed heuristics is to maximize the LBC while also minimizing the number of cuts.

B. Proposed Heuristic Algorithms

The heuristics of the final algorithm is developed incrementally from the basic greedy algorithm. Each algorithm attempts to construct a partition list L that balances computational load while limiting mesh duplication.

- 1) Greedy No-Split Distribution The first algorithm assigns meshes in decreasing size order to the processor with the largest remaining capacity. This approach produces no mesh splits but can suffer from imbalance when mesh

Algorithm 1 Greedy No-Split Distribution

Input: Meshes M , sizes S , processors P

Output: Partition list L

Sort meshes such that $S_1 \geq \dots \geq S_N$ $g \leftarrow \frac{\sum S_j}{P}$ **for** $i \leftarrow 1$ **to** P **do**

$B_i \leftarrow \emptyset$ $r_i \leftarrow g$

for $i \leftarrow 1$ **to** $\min(P, N)$ **do**

 assign M_i to B_i $r_i \leftarrow r_i - S_i$

for $j \leftarrow P + 1$ **to** N **do**

$k \leftarrow \arg \max_i r_i$ assign M_j to B_k $r_k \leftarrow r_k - S_j$

return L

sizes vary widely.

- 2) Greedy Distribution with Bin Merging The second algorithm introduces a tolerance parameter τ . If a processor exceeds the allowed imbalance, bins are merged. When sets of meshes are combined, each mesh in the combined set is split in equal pieces, and each processor receives one piece of each mesh. The number of pieces created is equal to the number of processors participating in the split.

- 3) Iterative Distribution with Selective Exclusion

Given the algorithms 1 & 2, the final heuristic can be constructed by combining the advantages of greedy partitioning and iterative bin rebalancing. The key idea is to first remove meshes that can already be well-balanced across processors, and then focus computational effort only on the remaining difficult cases.

The greedy algorithm (Algorithm 1) is therefore used as a preprocessing step to identify meshes that can be assigned cleanly without requiring any cuts. These meshes become *singletons*, while the remaining meshes form groups that require further balancing.

The complete method consists of two phases.

Phase 1: Singleton Creation When the number of meshes is less than or equal to the number of processing elements, greedy partitioning produces bins B_i whose cardinality is either one or zero. In other words, each processor receives either a single mesh or none.

Algorithm 2 Greedy Distribution with Bin Merging

Input: Meshes M , sizes S , processors P , tolerance τ **Output:** Partition list L Sort meshes in decreasing size $g \leftarrow \frac{\sum S_j}{P}$ $\delta \leftarrow \tau g$ perform greedy assignment as in Algorithm 1**while true do** $i_{max} \leftarrow \arg \max_i r_i$ $i_{min} \leftarrow \arg \min_i r_i$ **if** $r_{i_{min}} \leq -\delta$ **then** $\perp$ break merge $B_{i_{min}}$ into $B_{i_{max}}$ construct L **return** L

The first phase iteratively evaluates greedy partitioning while progressively removing meshes from consideration. After each partitioning, the worst loaded processor is identified, and if it is above the tolerance, the largest mesh in that processor is removed from the current set and saved for later. This is repeated until all processors are within the tolerance. The algorithm then proceeds to the second phase, where the remaining meshes are handled collectively using the grouping algorithm with bin merging.

Phase 2: Rebalancing the Greedy Partitioning

The second phase begins from the greedy partitioning of the remaining meshes. This initial state provides a reasonable but not necessarily balanced distribution.

The algorithm then iteratively improves the load balance until the load balancing coefficient (LBC) satisfies a prescribed tolerance. At each iteration, the bins with the maximum load and minimum load are identified and merged into a temporary combined set of meshes. The meshes in this combined set are then repartitioned evenly across the participating bins so that their loads become identical after redistribution.

This process is repeated until the target balance criterion is satisfied.

The listing of the complete algorithm is given below.

Algorithm 3 Two-Phase Heuristic Partitioning

Input: Meshes M , processors P , tolerance ϵ **Output:** Bins $B_1 \dots B_P$ **Phase 1: Singleton Creation;** $M_r \leftarrow M$;**while true do** Perform greedy partitioning on M_r ; **if** LBC for most loaded processor above tolerance **then** $\perp$ Mark largest mesh as needing further processing; Remove from M_r ; **else** $\perp$ break;**Phase 2: Rebalancing;**Initialize bins using greedy partitioning of M_r ;**while** $LBC < \epsilon$ **do** Identify bins with maximum and minimum load; Combine their mesh sets; Evenly repartition the combined meshes
 $\perp$ across those bins;Return all bins including singleton assignments;

C. Resulting Properties

The final configuration of bins B_i satisfies the following constraints

- 1) For any bins B_i and B_j ,

$$B_i \cap B_j \neq \emptyset$$

implies

$$B_i \cap B_j = B_i \cap B_k \quad \forall k \text{ such that } B_i \cap B_k \neq \emptyset$$

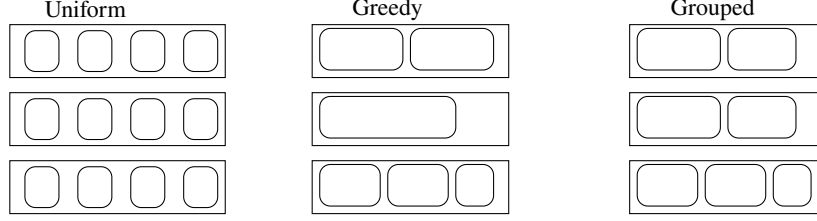


Fig. 2 Conceptual comparison of partitioning strategies. Uniform partitioning splits every mesh across processors, greedy partitioning avoids splits but can lead to imbalance, and the proposed heuristic balances load while minimizing splits.

2) The bins collectively cover the entire mesh set

$$\bigcup_{i=1}^P B_i = M$$

The first constraint is necessary for the execution of layered or wavefronts in the simulation code, where all partitions of a mesh M_j must be processed synchronously by every processing element that owns a portion of that mesh. After the synchronized phase completes, the singleton meshes produced during Phase 1 can be executed asynchronously, improving overall efficiency.

D. Discussion

These three algorithms represent progressively more robust heuristics. The first performs a simple greedy assignment, the second introduces bin merging to control imbalance, and the third improves stability by isolating and redistributing near-equal meshes. In practice, we observe that overset CFD simulations frequently contain a small number of very large meshes together with many smaller ones. Standard greedy methods tend to place these large meshes early, which can dominate the load distribution and produce poor balance. The selective exclusion strategy helps mitigate this effect by temporarily removing dominant meshes, allowing the remainder of the workload to equilibrate before they are reintroduced. Since partitioning is performed only once as a preprocessing step, the computational cost of these heuristics is negligible relative to the overall simulation.

Figure 2 illustrates the conceptual difference between uniform partitioning, greedy partitioning, and the proposed heuristic. Uniform partitioning produces a perfect load balance but introduces a large number of mesh splits. Greedy partitioning avoids splits, but can lead to a significant load imbalance. The proposed approach combines both ideas by first extracting singleton meshes and then redistributing the remaining meshes through controlled bin merging.

III. Canonical Tests

The time per iteration T_{iter} for a parallel CFD code on GPUs is composed of computational work (including ghost cell overhead) and communication costs. For a total grid size N partitioned across P GPUs, let the local domain size be $n = N/P$.

A. Ideal Parallel Time

The ideal time assumes zero communication and zero redundant work, accounting only for the GPU saturation effect:

$$T_{ideal} = \frac{I}{P_{peak}} \left(\frac{N}{P} + n_{1/2} \right) \quad (4)$$

where I is the number of iterations, P_{peak} is the peak throughput, and $n_{1/2}$ is the saturation constant, i.e. the number of cells required for a single GPU to achieve at least half the achievable throughput.

B. Actual Parallel Time

The actual time accounts for the computational cost of ghost cells (C_g) (additional needed for communication at interfaces) and the network overhead defined by communication constant relating to bandwidth (β) and latency (α). The

amount of surface communication is proportional to $(\frac{N}{P})^{2/3}$, where the 2/3 factor arises because communications are on the surface of a partition. So given there are N cells in 3D, $N^{1/3}$ is notional length scale and $N^{2/3}$ is a surface scale.

$$T_{total} = I \left[\frac{\frac{N}{P} + C_g \left(\frac{N}{P}\right)^{2/3} + n_{1/2}}{P_{peak}} + \beta \left(\frac{N}{P}\right)^{2/3} + \alpha \right] \quad (5)$$

C. Parallel Efficiency Calculation

Parallel efficiency ϵ is defined as the ratio of the ideal time to the actual total time:

$$\epsilon(P) = \frac{T_{ideal}}{T_{total}} \quad (6)$$

By substituting the components and canceling the constant I , we obtain:

$$\epsilon(P) = \frac{\frac{N}{P} + n_{1/2}}{\left(\frac{N}{P} + n_{1/2}\right) + C_g \left(\frac{N}{P}\right)^{2/3} + P_{peak} \left[\beta \left(\frac{N}{P}\right)^{2/3} + \alpha\right]} \quad (7)$$

To highlight the scaling loss, the expression is normalized by the ideal computational term:

$$\epsilon(P) = \left[1 + \frac{(C_g + \beta P_{peak}) \left(\frac{N}{P}\right)^{2/3} + \alpha P_{peak}}{\frac{N}{P} + n_{1/2}} \right]^{-1} \quad (8)$$

The parameters used in the equations above are shown in Table 1 The efficiency expression reveals three primary

Table 1 System Parameters

Parameter	Value
$P_{peak}(throughput)$	100M cells/second
Bandwidth	200 Gb/s
$n_{1/2}$	2 million cells
n_{fields}	5
β (comm factor)	$n_{fields} \times 8/\text{bandwidth}$
C_g (number of ghost layers)	1
α (latency)	5×10^{-6} seconds

mechanisms for the breakdown of scaling as P increases:

- 1) **Surface-to-Volume Penalty:** The term $(\frac{N}{P})^{2/3}$ in the numerator decreases slower than the $\frac{N}{P}$ term in the denominator. As $P \rightarrow \infty$, the communication and ghost-work ratio grows relative to useful computation.
- 2) **Hardware Under-saturation:** When $\frac{N}{P} < n_{1/2}$, the GPU no longer has sufficient threads to hide instruction latency, effectively hitting a performance floor while communication costs continue to accumulate.
- 3) **Latency Bottleneck:** The constant αP_{peak} represents a fixed overhead that remains even as the local problem size vanishes, causing efficiency to drop toward zero.

The equations shown above are for a single mesh with N cells distributed over P processors. The time-to-solution for an arbitrary mesh distribution with multiple meshes must be determined by computing the time for each mesh individually. First, the distribution is divided into mesh groups, which are groups of meshes across processors that can be executed together. The time for each mesh group to execute can then be determined by aggregating the time spent for each mesh. Groups are then optimally ordered to avoid overlap, and processor times are accumulated sequentially, accounting for the delay before all processors in a mesh group are ready to begin. Once all processor times have been determined, the maximum time is used to calculate the parallel efficiency.

To effectively test algorithm performance over varying test cases, the following strategies were used:

- 1) Heatmap plots to analyze performance over test cases with a varying number of processors as well as meshes. For each pair of values for the number of PEs and meshes, the performance is determined by averaging parallel efficiency (ϵ) over test cases with mesh sizes selected according to a probability distribution (Figure 3).
- 2) Parallel Efficiency vs Mesh-processor ratio plots to show how an algorithm does when there are more/less meshes than PEs (Figure 4(a)).
- 3) Parallel Efficiency vs allowed load-balancing tolerance to show how the efficiency varies with rigid and flexible constraints (Figure 4(b)).

The algorithms evaluated are:

- 1) Uniform - all meshes are divided evenly among all processors
- 2) Greedy - meshes are distributed sequentially to the least loaded processor
- 3) Grouping - a heuristic is used to determine groups of meshes that are split among groups of processors
- 4) Singleton-Group - a heuristic identifies uncut meshes, then adds remaining meshes in groups

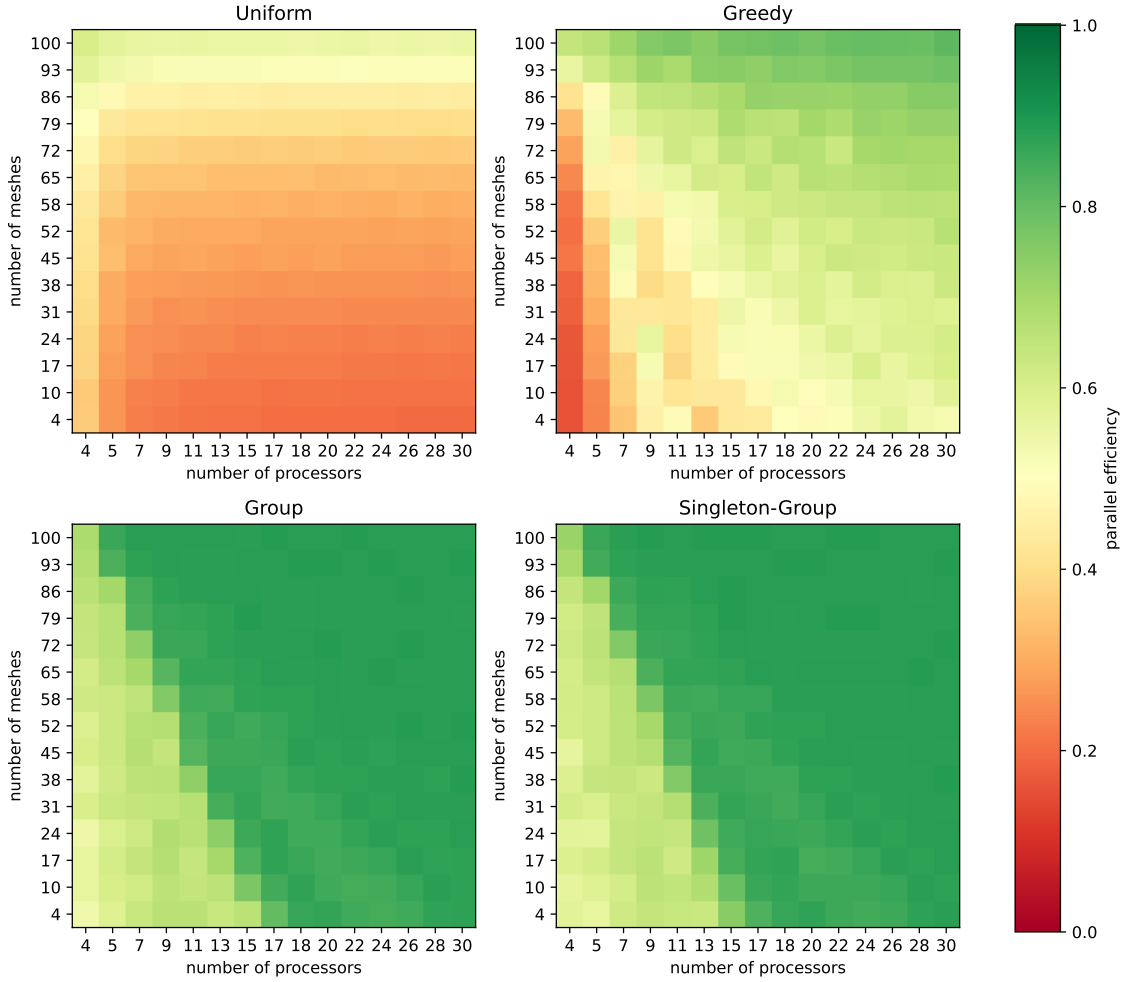


Fig. 3 Heatmaps for each algorithm: For each pair of processor and mesh counts, 20 tests with random mesh sizes are performed, and parallel efficiency is averaged to determine the data value.

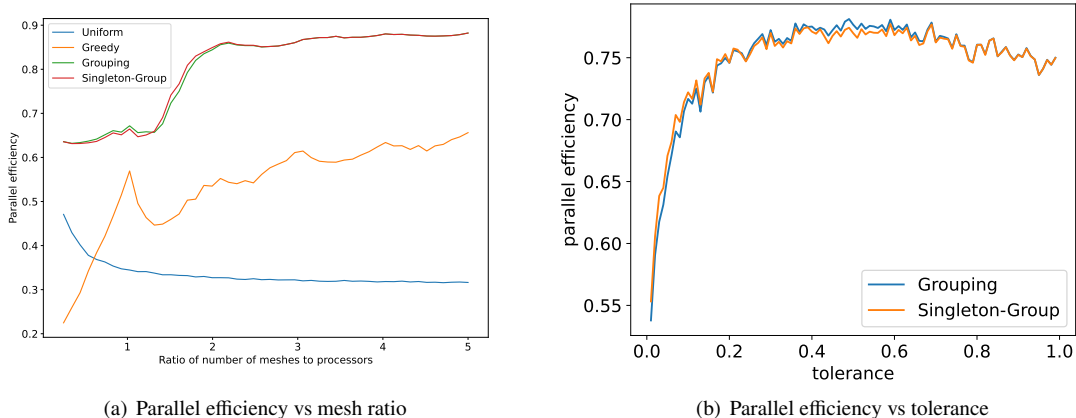


Fig. 4 Parallel efficiency vs mesh ratio and tolerance: For each ratio, parallel efficiency is averaged over 20 random mesh size tests for each processor count, which ranges from 4 to 32 in steps of 2.

It is evident from the plots presented above that the grouping and singleton-group algorithm are significantly more efficient than the uniform partitioning and greedy algorithms.

IV. Drag prediction workshop cases

The AIAA Drag Prediction Workshops (DPW) are periodic computational fluid dynamics (CFD) validation and verification exercises. The DPW working group publicly releases the meshes used in each workshop. To assess the applicability of the proposed method, the overset meshes from the DPW5 workshop were downloaded from https://dpw.larc.nasa.gov/DPW5/overset_grids.REV01. Algorithm 3 (Singleton-Group) was then applied to partition the coarse, medium, and fine grid systems into 4, 8, and 12 processing elements (PEs), respectively. From Table 2, it is evident that the algorithm behaves consistently with its design objectives. Larger meshes are preferentially grouped and subjected to a greater number of partitioning cuts to expose sufficient parallelism, while smaller meshes are either left intact or partitioned minimally to avoid unnecessary fragmentation and communication overhead. Across all three cases, the resulting load imbalance remains below 10%, demonstrating effective workload distribution and confirming the scalability and robustness of the proposed approach.



(a) XV-15 aircraft https://en.wikipedia.org/wiki/Bell_XV-15 (b) Dragonfly https://en.wikipedia.org/wiki/Dragonfly_Titan_space_probe

Fig. 5 XV-15 aircraft model contains 9 bodies : aircraft, two pylons and 6 rotor blades. Dragonfly aircraft model contains 25 bodies: airframe and 24 rotor blades.

Mesh Information						
Case	Dimensions	Cells (M)	Total (M)	Mesh distribution	Cells per PE (M)	Imbalance
Coarse (4 PEs)	M_1 : (97,97,145)	1.36	18.62	$L = \begin{bmatrix} 4 & 2 & 5 & 8 \\ 4 & 2 & 5 & 6 \\ 4 & 2 & 5 & 7 \\ 9 & 3 & 1 & \end{bmatrix}$	$\begin{bmatrix} 4.91 \\ 4.62 \\ 4.62 \\ 4.47 \end{bmatrix}$	5.3%
	M_2 : (385,97,145)	5.42				
	M_3 : (97,193,145)	2.71				
	M_4 : (385,49,145)	2.74				
	M_5 : (385,97,145)	5.42				
	M_6 : (97,7,145)	0.10				
	M_7 : (97,7,145)	0.10				
	M_8 : (385,7,145)	0.39				
	M_9 : (385,7,145)	0.39				
Medium (8 PEs)	M_1 : (129,129,193)	3.21	43.32	$L = \begin{bmatrix} 5 & 2 & 9 \\ 5 & 2 & 8 \\ 5 & 2 \\ 5 & 2 & 7 \\ 5 & 2 \\ 1 & 4 & 3 & 6 \\ 1 & 4 & 3 \\ 1 & 4 & 3 \end{bmatrix}$	$\begin{bmatrix} 5.80 \\ 5.80 \\ 5.10 \\ 5.28 \\ 5.10 \\ 5.53 \\ 5.34 \\ 5.34 \end{bmatrix}$	6.67%
	M_2 : (513,129,193)	12.77				
	M_3 : (129,257,193)	6.40				
	M_4 : (513,65,193)	6.44				
	M_5 : (513,129,193)	12.77				
	M_6 : (129,7,193)	0.17				
	M_7 : (129,7,193)	0.17				
	M_8 : (513,7,193)	0.69				
	M_9 : (513,7,193)	0.69				
Fine (12 PEs)	M_1 : (193,193,289)	10.76	143.47	$L = \begin{bmatrix} 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 1 & 4 & 9 & 5 & 2 \\ 8 & 3 \\ 8 & 3 \end{bmatrix}$	$\begin{bmatrix} 11.97 \\ 12.36 \\ 11.97 \\ 11.97 \\ 12.36 \\ 11.97 \\ 11.97 \\ 11.97 \\ 11.97 \\ 11.51 \\ 11.51 \end{bmatrix}$	3.24 %
	M_2 : (769,193,289)	42.89				
	M_3 : (193,385,289)	21.47				
	M_4 : (769,97,289)	21.56				
	M_5 : (769,193,289)	42.89				
	M_6 : (193,7,289)	0.39				
	M_7 : (193,7,289)	0.39				
	M_8 : (769,7,289)	1.56				
	M_9 : (769,7,289)	1.56				

Table 2 DPW mesh sizes, total mesh sizes (millions of cells), partition list L and imbalance. The partition list L has as many rows as there are PEs. Each rows show the bin B (list of meshes) that this PE is processing. If a mesh is present only in one row, it is considered a singleton. Further the bins satisfy the constraints listed in Section II.C

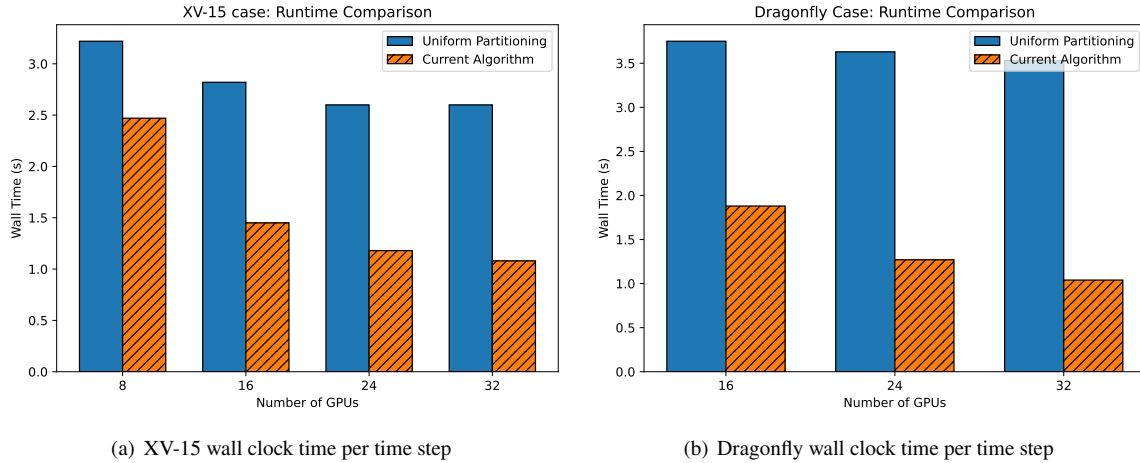


Fig. 6 Comparison of uniform partitioning (original implementation) and current algorithm (singleton-group) for large production cases in Helios clearly demonstrating significant speedup in wall clock time.

V. Algorithm use in CREATE™-AV Helios

The developed algorithm is available open source on GitHub and was utilized by the Helios team to partition the overset mesh system utilized by the RAPIDUS [4] flow solver. Two cases shown in Figure 5 are analyzed. The first is the Bell XV-15 aircraft with 9 different meshes, one for the fuselage (45.6 million cells), two pylons (2 million cells each) and 6 rotor blades (6 million cells each). The second one is the Dragonfly aircraft that is set to fly on Titan as part of an upcoming NASA mission. This aircraft model had 25 meshes, one for the airframe (16 million cells) and 24 blades (5 million cells each) distributed over 8 rotors. The first case was executed on 8, 16, 24, 32 H100 GPUs and second one was executed on 16, 24, and 32 GPUs. Figure 6 shows the performance comparison reported as time taken per time step for both of these cases. It is evident that the current algorithm performs superior to uniform partitioning approach and shows better scaling with increase in the number of GPUs. The uniform partitioning algorithm saturates because of lower mesh counts and increased communication at the 16 GPU level for both cases. The proposed algorithm continues to show improved scaling and provides about a factor of 2.5 to 3.5 improvement in run time compared to the uniform algorithm on larger GPU counts. Since these cases are run for large number of time steps, the improvement is significant for analysts in terms of time-savings and energy consumption to obtain engineering results.

VI. Conclusions

A heuristic algorithm for load-balancing mesh partitioning for GPU based systems was developed and published on github. Usage of the algorithm in a modern high performance computing code Helios indicated significant gain in performance. Specific observations are as follows:

- 1) Results of canonical evaluation of algorithm by parametrically varying the number of meshes, their sizes and the number of PEs for mesh distribution showed that the grouping algorithm and singleton-group variants outperform the greedy and uniform distribution algorithms.
- 2) Applications to actual mesh systems from DPW5 workshop series showed expected results with low imbalance and no cuts applied to small meshes.
- 3) Execution within CREATE™-AV Helios for two large production cases showed improvements in time-to-solution by factors of 2.5-3.5 for the flow solver RAPIDUS at larger GPU counts.

Future enhancements to the algorithm will be explored to eliminate the undivided (singleton) mesh phase in the current implementation, allowing new processor groups and computational wave fronts to be formed during a second pass. This extension is expected to further improve flexibility and load balance for challenging configurations.

Acknowledgments

The author acknowledges Dr. Shirzad Hosseinverdi of the HPCMP CREATE-AV program for incorporating the developed algorithm into the RAPIDUS [4] code base and providing simulation time comparisons with the uniform partitioning algorithm.

References

- [1] Ventura Diaz, P., and Yoon, S., "High-fidelity computational aerodynamics of multi-rotor unmanned aerial vehicles," *2018 AIAA Aerospace Sciences Meeting*, 2018, p. 1266.
- [2] Jackson, C. W., Appelhans, D., Derlaga, J. M., and Buning, P. G., "GPU implementation of the OVERFLOW CFD code," *AIAA SCITECH 2024 Forum*, 2024, p. 0042.
- [3] Walden, A., Nielsen, E., and Nastac, G., "Accelerating FUN3D solutions using GPU hardware," *FUN3D GPU Training*, 2020.
- [4] Premaratne, P. D., Hosseinverdi, S., and Jude, D., "Verification and Validation of a Performance-Portable Unstructured Navier-Stokes Solver (RAPIDUS)," *AIAA SCITECH 2025 Forum*, 2025, p. 0892.
- [5] Krakos, J., Gomes, P., Ho, J., Mara, M., Sáez, G., Loseille, A., Economon, T., and Alonso, J. J., "GPU-based and Adaptive Solution Technology for the 5th AIAA High Lift Prediction Workshop," *AIAA SCITECH 2025 Forum*, 2025, p. 0496.
- [6] Jia, R., Kamel, M. S., Wu, C., and Agrawal, B., "Ansys Fluent HPC for Large-Scale CFD Simulations," *AIAA SCITECH 2025 Forum*, 2025, p. 1950.
- [7] Wissink, A. M., and Meakin, R. L., "On parallel implementations of dynamic overset grid methods," *Proceedings of the 1997 ACM/IEEE conference on Supercomputing*, 1997, pp. 1–21.
- [8] Sherer, S. E., and Garmann, D. J., "Automatic Partitioning of Structured Overset Grids for Use with High-Order Algorithms," *2018 AIAA Aerospace Sciences Meeting*, 2018, p. 1405.
- [9] Sherer, S. E., and Garmann, D. J., "Further Development and Performance Assessment of Automatic Partitioning Algorithms for Structured Overset Grids," *AIAA Scitech 2019 Forum*, 2019, p. 1987.